

EXPERIMENT-16:

Compare different types Classification Algorithms and evaluate their performance.

Code:

```
# =====  
  
# Compare Different Classification Algorithms  
# Dataset: heart.csv  
# =====  
  
# Step 1: Upload Dataset  
from google.colab import files  
uploaded = files.upload()  
  
# Step 2: Import Libraries  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.naive_bayes import GaussianNB  
from sklearn.svm import SVC  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score  
  
# Step 3: Load Dataset  
df = pd.read_csv("heart.csv")  
print("Dataset Shape:", df.shape)  
print(df.head())  
  
# Step 4: Features and Target  
X = df.drop("target", axis=1)  
y = df["target"]  
  
# Step 5: Split Dataset  
X_train, X_test, y_train, y_test = train_test_split(
```

```

X,
y,
test_size=0.20,
random_state=42
)
# Step 6: Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Step 7: Define Models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}
# Step 8: Train and Evaluate
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    results[name] = acc
# Step 9: Display Accuracy
print("\nAccuracy Comparison\n")
for name, acc in results.items():
    print(f'{name:25s} : {acc*100:.2f}%')

```

```

# Step 10: Plot Results

plt.figure(figsize=(9,5))

plt.bar(results.keys(), results.values())

plt.xticks(rotation=25)

plt.ylabel("Accuracy")

plt.title("Classification Algorithm Comparison")

plt.show()

# Step 11: Best Model

best_model = max(results, key=results.get)

print("\nBest Algorithm:", best_model)

print("Accuracy:", round(results[best_model]*100,2),"%")

```

Output:

```

... Choose Files heart.csv
heart.csv(text/csv) - 38114 bytes, last modified: 8/5/2026 - 100% done
Saving heart.csv to heart.csv
Dataset Shape: (1025, 14)
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52   1   0     125    212   0         1     168     0       1.0     2
1   53   1   0     140    203   1         0     155     1       3.1     0
2   70   1   0     145    174   0         1     125     1       2.6     0
3   61   1   0     148    203   0         1     161     0       0.0     2
4   62   0   0     138    294   1         1     106     0       1.9     1

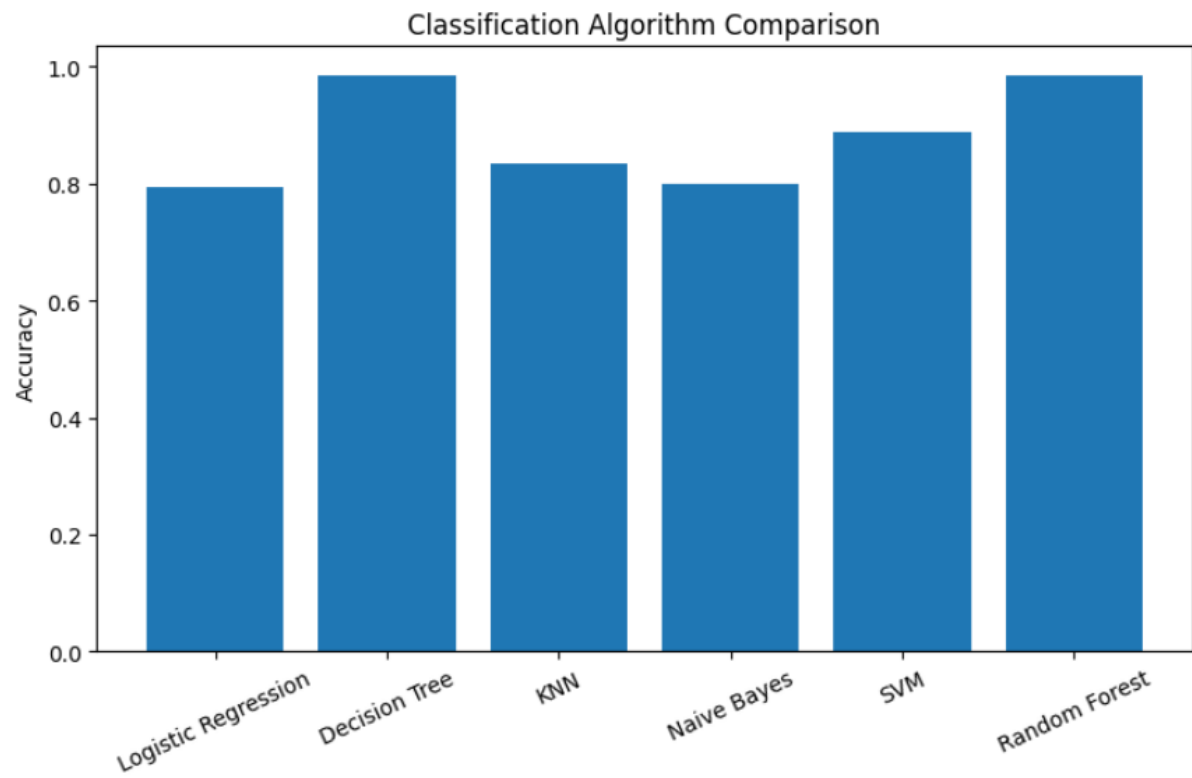
   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

Accuracy Comparison

Logistic Regression      : 79.51%
Decision Tree            : 98.54%
KNN                      : 83.41%
Naive Bayes              : 80.00%
SVM                      : 88.78%
Random Forest            : 98.54%

```

...



Best Algorithm: Decision Tree
Accuracy: 98.54 %